

1. Processes

A process is a program in execution. During its lifetime a process moves through the states new, ready, running, waiting (blocked) and terminated. The scheduler moves a process from ready to running; an I/O request moves it from running to waiting.

The Process Control Block (PCB) stores everything the OS needs to manage a process: process id, process state, program counter, CPU registers, scheduling information (priority), memory-management information (page tables), accounting information and the list of open files.

A context switch saves the state of the running process into its PCB and loads the saved state of the next process. Context switching is pure overhead because no useful work is done during the switch.

2. CPU Scheduling

Scheduling criteria: CPU utilisation, throughput, turnaround time, waiting time and response time.

Turnaround time = completion time - arrival time. Waiting time = turnaround time - burst time.

- First-Come, First-Served (FCFS): non-preemptive; simple, but suffers from the convoy effect when a long job delays short ones.
- Shortest Job First (SJF): gives the minimum average waiting time for a given set of processes, but requires knowing the next CPU burst length. Its preemptive version is Shortest Remaining Time First (SRTF).
- Round Robin (RR): each process gets a fixed time quantum and is then preempted and put at the end of the ready queue. A very large quantum makes RR behave like FCFS; a very small quantum causes too many context switches.
- Priority scheduling: the highest priority process runs first. Starvation of low-priority processes is solved by aging, which gradually increases the priority of waiting processes.

3. Deadlocks

A deadlock is a situation where a set of processes are blocked because each process holds a resource and waits for a resource held by another process in the set.

Coffman conditions - a deadlock can occur only if all four hold simultaneously:

1. Mutual exclusion: at least one resource is non-sharable.
2. Hold and wait: a process holds at least one resource while waiting for others.
3. No preemption: resources cannot be forcibly taken from a process.
4. Circular wait: a circular chain of processes exists, each waiting for a resource held by the next.

Deadlock handling strategies are prevention (break one of the four conditions), avoidance, detection with recovery, and ignoring the problem (the ostrich approach).

Banker's algorithm (deadlock avoidance): each process declares its maximum need in advance. A request is granted only if the resulting state is safe, meaning there exists a sequence in which every process can obtain its maximum need and finish. $\text{Need} = \text{Max} - \text{Allocation}$.

4. Memory Management and Paging

Paging divides physical memory into fixed-size frames and logical memory into pages of the same size. The page table maps each page number to a frame number, which removes external fragmentation (internal fragmentation can still occur in the last page).

A logical address is split into a page number p and a page offset d . The Translation Lookaside Buffer (TLB) is a small, fast associative cache of recent page-table entries that avoids a second memory access for most translations.

A page fault occurs when a referenced page is not in main memory; the OS loads it from disk, possibly replacing another page.

Page replacement algorithms:

- FIFO replaces the oldest page. FIFO suffers from Belady's anomaly: increasing the number of frames can increase the number of page faults.
- Optimal (OPT) replaces the page that will not be used for the longest time in the future. It gives the lowest fault rate but cannot be implemented in practice.
- Least Recently Used (LRU) replaces the page that has not been used for the longest time. LRU does not suffer from Belady's anomaly.

5. Process Synchronization

The critical-section problem requires a solution that satisfies three conditions:

- Mutual exclusion: only one process executes in its critical section at a time.
- Progress: if no process is in its critical section, the choice of the next process to enter cannot be postponed indefinitely.
- Bounded waiting: there is a limit on how many times other processes may enter their critical sections after a process has requested entry.

A semaphore is an integer variable accessed only through two atomic operations: wait (P), which decrements the value and blocks if it becomes negative, and signal (V), which increments the value and wakes a blocked process. A binary semaphore (mutex) takes only the values 0 and 1; a counting semaphore can take any non-negative value.

In the producer-consumer (bounded buffer) problem, a mutex protects the buffer, a semaphore 'empty' counts free slots and a semaphore 'full' counts filled slots.